

2장 데이터 입·출력 구현



440063



034 스키마

(A)

23.4, 20.10, 필기 25.5, 21.3, 20.9

스키마(Schema)는 데이터베이스의 구조와 제약조건에 관한 전반적인 명세를 기술한 것이다.

종류	내용
23.4 외부 스키마	사용자나 응용 프로그램이 각 개인의 입장에서 필요로 하는 데이터베이스의 논리적 구조를 정의한 것
23.4, 필기 25.5, ... 개념 스키마	- 데이터베이스의 전체적인 논리적 구조 - 모든 응용 프로그램이나 사용자들이 필요로 하는 데이터를 종합한 조직 전체의 데이터베이스로, 하나만 존재함
23.4, 필기 20.9 내부 스키마	- 물리적 저장장치의 입장에서 본 데이터베이스 구조 - 실제로 저장될 레코드의 형식, 저장 데이터 항목의 표현 방법, 내부 레코드의 물리적 순서 등을 나타냄

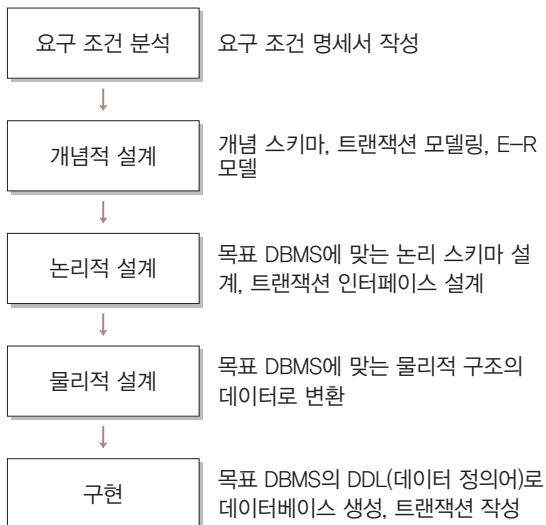
340081



035 데이터베이스 설계 순서

(A)

23.7, 20.7



340082



036 개념적 설계

(A)

21.4, 필기 23.2, 22.4

- 개념적 설계(정보 모델링, 개념화)는 정보의 구조를 얻기 위하여 현실 세계의 무한성과 계속성을 이해하고, 다른 사람과 통신하기 위하여 현실 세계에 대한 인식을 추상적 개념으로 표현하는 과정이다.
- 개념적 설계에서는 개념 스키마 모델링과 트랜잭션 모델링을 병행 수행한다.
- 개념적 설계에서는 요구 분석에서 나온 결과인 요구 조건 명세를 DBMS에 독립적인 E-R 다이어그램으로 작성한다.

340083



037 논리적 설계

(A)

21.4, 필기 24.7, 22.7, 20.6

- 논리적 설계(데이터 모델링)는 현실 세계에서 발생하는 자료를 컴퓨터가 이해하고 처리할 수 있는 물리적 저장장치에 저장할 수 있도록 변환하기 위해 특정 DBMS가 지원하는 논리적 자료 구조로 변환(mapping)시키는 과정이다.
- 개념 세계의 데이터를 필드로 기술된 데이터 타입과 이 데이터 타입들 간의 관계로 표현되는 논리적 구조의 데이터로 모델화한다.
- 개념적 설계가 개념 스키마를 설계하는 단계라면, 논리적 설계에서는 개념 스키마를 평가 및 정제하고 DBMS에 따라 서로 다른 논리적 스키마를 설계하는 단계이다.



038 물리적 설계

(A)

- 물리적 설계(데이터 구조화)는 논리적 설계에서 논리적 구조로 표현된 데이터를 디스크 등의 물리적 저장장치에 저장할 수 있는 물리적 구조의 데이터로 변환하는 과정이다.
- 물리적 설계에서는 다양한 데이터베이스 응용에 대해 처리 성능을 얻기 위해 데이터베이스 파일의 저장 구조 및 액세스 경로를 결정한다.
- 저장 레코드의 형식, 순서, 접근 경로, 조회 집중 레코드 등의 정보를 사용하여 데이터가 컴퓨터에 저장되는 방법을 묘사한다.



039 데이터 모델

(A)

- 데이터 모델은 현실 세계의 정보들을 컴퓨터에 표현하기 위해서 단순화, 추상화하여 체계적으로 표현한 개념적 모형이다.
- 데이터 모델에 표시할 요소

요소	내용
21.4, 필기 24.5, 23.2, 20.9 구조(Structure)	논리적으로 표현된 개체 타입들 간의 관계로서 데이터 구조 및 정적 성질 표현
21.4, 필기 25.8, 24.5, 23.2, ... 연산(Operation)	데이터베이스에 저장된 실제 데이터를 처리하는 작업에 대한 명세로서 데이터베이스를 조작하는 기본 도구
21.4, 필기 24.5, 23.2, 20.9 제약 조건(Constraint)	데이터베이스에 저장될 수 있는 실제 데이터의 논리적인 제약 조건



040 E-R 다이어그램

(B)

기호	기호 이름	의미
필기 25.5, 24.7, 24.5, 23.7, 22.7, ... 	사각형	개체(Entity) 타입
필기 24.7, 24.5, 23.7, 21.5, 21.3, ... 	마름모	관계(Relationship) 타입
필기 25.5, 24.7, 24.5, 23.7, 21.5, ... 	타원	속성(Attribute)
필기 22.3 	이중 타원	다중값 속성(복합 속성)
	밑줄 타원	기본키 속성
	복수 타원	복합 속성 예 성명은 성과 이름으로 구성
	관계	1:1, 1:N, N:M 등의 개체 간 관계에 대한 대응수를 선 위에 기술함
필기 24.7, 24.5, 21.5, 21.3, 20.9, ... 	선, 링크	개체 타입과 속성을 연결

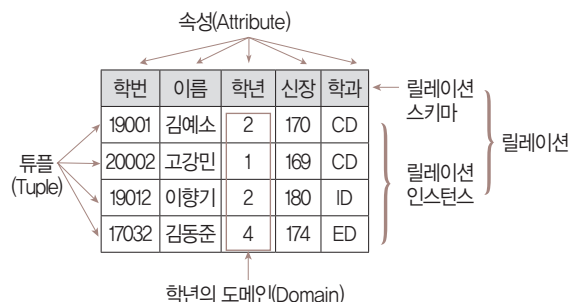


041 관계형 데이터베이스의 릴레이션 구조

(A)

릴레이션(Relation)은 데이터들을 표(Table)의 형태로 표현한 것으로, 구조를 나타내는 릴레이션 스키마와 실제 값들인 릴레이션 인스턴스로 구성된다.

〈학생〉 릴레이션



- 릴레이션 인스턴스 : 데이터 개체를 구성하고 있는 속성들에 데이터 타입이 정의되어 구체적인 데이터 값을 가진 것을 의미함

340071

25.11, 25.4, 24.7, 23.4, 21.4, 필기 25.5, 23.2, 22.4, 22.3, 21.3, 20.9, 20.8



042 튜플



- 튜플(Tuple)은 릴레이션을 구성하는 각각의 행을 말한다.
- 튜플은 속성의 모임으로 구성된다.
- 파일 구조에서 레코드와 같은 의미이다.
- 튜플의 수를 카디널리티(Cardinality) 또는 기수, 대응수라고 한다.

340092

25.7, 25.4, 24.7, 21.4, 필기 25.5, 23.2, 22.3, 21.5, 21.3, 20.9, 20.8



043 속성



- 속성(Attribute)은 데이터베이스를 구성하는 가장 작은 논리적 단위이다.
- 파일 구조상의 데이터 항목 또는 데이터 필드에 해당된다.
- 속성은 개체의 특성을 기술한다.
- 속성의 수를 디그리(Degree) 또는 차수라고 한다.

340093

25.4, 필기 25.8, 21.3, 20.6



044 도메인



- 도메인(Domain)은 하나의 애트리뷰트가 취할 수 있는 같은 타입의 원자(Atomic)값들의 집합이다.
- 도메인은 실제 애트리뷰트 값이 나타날 때 그 값의 합법 여부를 시스템이 검사하는 데에도 이용된다.
- 예 '성별' 애트리뷰트의 도메인은 "남"과 "여"로, 그 외의 값은 입력될 수 없다.

340094

24.10, 22.5, 필기 25.5, 22.7, 22.4, 20.6



045 후보키



- 후보키(Candidate Key)는 릴레이션을 구성하는 속성들 중에서 튜플을 유일하게 식별하기 위해 사용되는 속성들의 부분집합이다.
- 기본키로 사용할 수 있는 속성들을 말한다.
- 후보키는 유일성(Unique)과 최소성(Minimality)을 모두 만족시켜야 한다.

22.5, 필기 20.6

유일성 (Unique)

하나의 키 값으로 하나의 튜플만을 유일하게 식별할 수 있어야 함

22.5, 필기 20.6

최소성 (Minimality)

키를 구성하는 속성 하나를 제거하면 유일하게 식별할 수 없도록 꼭 필요한 최소의 속성으로 구성되어야 함

440076

24.10, 필기 22.7



046 대체키



- 대체키(Alternate Key)는 후보키가 둘 이상일 때 기본키를 제외한 나머지 후보키를 의미한다.
- 대체키를 보조키라고도 한다.

340097

24.10, 22.5, 필기 25.2, 24.5, 23.7, 22.7, 21.8, 20.9



047 슈퍼키



- 슈퍼키(Super Key)는 한 릴레이션 내에 있는 속성들의 집합으로 구성된 키를 말한다.
- 릴레이션을 구성하는 모든 튜플 중 슈퍼키로 구성된 속성의 집합과 동일한 값은 나타나지 않는다.
- 슈퍼키는 릴레이션을 구성하는 모든 튜플에 대해 유일성은 만족하지만, 최소성은 만족하지 못한다.



048 외래키



- 외래키(Foreign Key)는 다른 릴레이션의 기본키를 참조하는 속성 또는 속성들의 집합을 의미한다.
- 한 릴레이션에 속한 속성 A와 참조 릴레이션의 기본키인 B가 동일한 도메인 상에서 정의되었을 때의 속성 A를 외래키라고 한다.
- 외래키로 지정되면 참조 릴레이션의 기본키에 없는 값은 입력할 수 없다.



049 무결성



- 무결성(Integrity)은 데이터베이스에 저장된 데이터 값과 그것이 표현하는 현실 세계의 실제값이 일치하는 정확성을 의미한다.
- 개체 무결성: 기본 테이블의 기본키를 구성하는 어떤 속성도 Null 값이나 중복값을 가질 수 없다는 규정
- 참조 무결성: 외래키 값은 Null이거나 참조 릴레이션의 기본키 값과 동일해야 함. 즉 릴레이션은 참조할 수 없는 외래키 값을 가질 수 없다는 규정
- 도메인 무결성: 주어진 속성 값이 정의된 도메인에 속한 값이어야 한다는 규정



050 관계대수



- 관계대수는 관계형 데이터베이스에서 원하는 정보와 그 정보를 검색하기 위해서 어떻게 유도하는가를 기술하는 절차적인 언어이다.
- 관계대수는 릴레이션을 처리하기 위해 연산자와 연산규칙을 제공하며, 피연산자와 연산 결과가 모두 릴레이션이다.
- 관계대수는 질의에 대한 해를 구하기 위해 수행해야 할 연산의 순서를 명시한다.



051 순수 관계 연산자



23.10, 필기 25.5, 24.5, 23.7, 23.2, 21.3, 20.8

Select

- 릴레이션에 존재하는 튜플 중에서 선택 조건을 만족하는 튜플의 부분집합을 구하여 새로운 릴레이션을 만드는 연산이다.
- 릴레이션의 행에 해당하는 튜플(Tuple)을 구하는 것이므로 수평 연산이라고도 한다.
- 기호: σ (시그마)

25.7, 23.10, 22.10, 22.7, 필기 24.5, 23.7, 23.2, 21.8, 21.5, 20.8

Project

- 주어진 릴레이션에서 속성 리스트(Attribute List)에 제시된 속성 값만을 추출하여 새로운 릴레이션을 만드는 연산이다.
- 릴레이션의 열에 해당하는 속성을 추출하는 것이므로 수직 연산자라고도 한다.
- 기호: π (파이)

23.10, 22.10, 필기 21.8, 21.5, 20.8

Join

- 공통 속성을 중심으로 두 개의 릴레이션을 하나로 합쳐서 새로운 릴레이션을 만드는 연산이다.
- Join의 결과는 Cartesian Product(교차곱)를 수행한 다음 Select를 수행한 것과 같다.
- 기호: $\bowtie$

25.11, 23.10, 20.10, 필기 24.5, 21.8, 21.5, 20.8

Division

- $X \supset Y$ 인 두 개의 릴레이션 $R(X)$ 와 $S(Y)$ 가 있을 때, R 의 속성이 S 의 속성값을 모두 가진 튜플에서 S 가 가진 속성을 제외한 속성만을 구하는 연산이다.
- 기호: $\div$



052 일반 집합 연산자



22.10

합집합(UNION)

- 두 릴레이션에 존재하는 튜플의 합집합을 구하되, 결과로 생성된 릴레이션에서 중복되는 튜플은 제거되는 연산이다.
- 합집합의 카디널리티는 두 릴레이션 카디널리티의 합보다 크지 않다.
- 기호 : $\cup$

교집합(INTERSECTION)

- 두 릴레이션에 존재하는 튜플의 교집합을 구하는 연산이다.
- 교집합의 카디널리티는 두 릴레이션 중 카디널리티가 적은 릴레이션의 카디널리티보다 크지 않다.
- 기호 : $\cap$

22.10

차집합(DIFFERENCE)

- 두 릴레이션에 존재하는 튜플의 차집합을 구하는 연산이다.
- 차집합의 카디널리티는 릴레이션 R의 카디널리티보다 크지 않다.
- 기호 : $-$

22.10, 필기 24.5, 24.2, 21.8, 21.5

교차곱(CARTESIAN PRODUCT)

- 두 릴레이션에 있는 튜플들의 순서쌍을 구하는 연산이다.
- 교차곱의 디그리(Degree)는 두 릴레이션의 디그리를 더한 것과 같고, 카디널리티(Cardinality)는 두 릴레이션의 카디널리티를 곱한 것과 같다.
- 기호 : $\times$



053 관계해석



- 관계해석(Relational Calculus)은 관계 데이터의 연산을 표현하는 방법이다.
- 관계 데이터 모델의 제안자인 코드(E. F. Codd)가 수학의 Predicate Calculus(술어 해석)에 기반을 두고 관계 데이터베이스를 위해 제안했다.
- 관계해석은 원하는 정보가 무엇이라는 것만 정의하는 비절차적 특성을 지닌다.
- 원하는 정보를 정의할 때는 계산 수식을 사용한다.



054 이상



- 이상(Anomaly)이란 데이터베이스 내에 데이터들이 불필요하게 중복되어 릴레이션 조작 시 예기치 않게 발생하는 곤란한 현상을 의미한다.
- 삽입 이상(Insertion Anomaly) : 테이블에 데이터를 삽입할 때 의도와는 상관없이 원하지 않은 값들로 인해 삽입할 수 없게 되는 현상
- 삭제 이상(Deletion Anomaly) : 테이블에서 튜플을 삭제할 때 의도와는 상관없는 값들도 함께 삭제되는, 즉 연쇄 삭제가 발생하는 현상
- 갱신 이상(Update Anomaly) : 테이블에서 튜플에 있는 속성 값을 갱신할 때 일부 튜플의 정보만 갱신되어 정보에 불일치성(Inconsistency)이 생기는 현상



055 함수적 종속

(A)

필기 24.5, 24.2, 21.8

함수적 종속(Functional Dependency)

어떤 테이블 R에서 X와 Y를 각각 R의 속성 집합의 부분 집합이라 하자. 속성 X의 값 각각에 대해 시간에 관계없이 항상 속성 Y의 값이 오직 하나만 연관되어 있을 때 Y는 X에 함수적 종속 또는 X가 Y를 함수적으로 결정한다고 하고, $X \rightarrow Y$ 로 표기한다.

22.7

완전 함수적 종속(Full Functional Dependency)

어떤 테이블 R에서 속성 Y가 다른 속성 집합 X 전체에 대해 함수적 종속이면서 속성 집합 X의 어떠한 진부분 집합 Z(즉, $Z \subset X$)에도 함수적 종속이 아닐 때 속성 Y는 속성 집합 X에 완전 함수적 종속이라고 한다.

22.7

부분 함수적 종속(Partial Functional Dependency)

어떤 테이블 R에서 속성 Y가 다른 속성 집합 X 전체에 대해 함수적 종속이면서 속성 집합 X의 임의의 진부분 집합에 대해 함수적 종속일 때, 속성 Y는 속성 집합 X에 부분 함수적 종속이라고 한다.

22.7, 필기 24.2, 22.3, 20.8, 20.6

이행적 함수적 종속(Transitive Functional Dependency)

$X \rightarrow Y$ 이고 $Y \rightarrow Z$ 일 때 $X \rightarrow Z$ 를 만족하는 관계를 의미한다.



056 정규화

(C)

- 정규화(Normalization)는 테이블의 속성들이 상호 종속적인 관계를 갖는 특성을 이용하여 테이블을 무손실 분해하는 과정이다.
- 정규화의 목적은 가능한 한 중복을 제거하여 삽입, 삭제, 갱신 이상의 발생 가능성을 줄이는 것이다.
- 정규형에는 제 1정규형(1NF), 제 2정규형(2NF), 제 3정규형(3NF), BCNF, 제 4정규형(4NF), 제 5정규형(5NF)이 있으며, 순서대로 정규화의 정도가 높아진다.



057 정규화 과정



아래의 <주문목록> 테이블을 가지고 정규화 과정을 살펴보자. <주문목록> 테이블의 기본키(Primary Key)는 제품번호이다.

<주문목록>

제품번호	제품명	재고수량	주문번호	고객번호	주소	주문수량
1001	모니터	2000	A345	100	서울	150
			D347	200	부산	300
1007	마우스	9000	A210	300	광주	600
			A345	100	서울	400
			B230	200	부산	700
1201	키보드	2100	D347	200	부산	300

• 제 1정규형(1NF; First Normal Form)

- 제 1정규형은 테이블 R에 속한 모든 속성의 도메인(Domain)이 원자 값(Atomic Value)만으로 되어 있는 정규형이다. 즉 테이블의 모든 속성 값이 원자 값으로만 되어 있는 정규형이다.
- <주문목록> 테이블에서는 하나의 제품에 대해 여러 개의 주문 관련 정보(주문번호, 고객번호, 주소, 주문수량)가 발생하고 있다. 따라서 <주문목록>테이블은 제 1정규형이 아니다.

예제 1 <주문목록> 테이블에서 반복되는 주문 관련 정보를 분리하여 제 1정규형으로 만드시오.

<주문목록>

제품번호	제품명	재고수량	주문번호	고객번호	주소	주문수량
1001	모니터	2000	A345	100	서울	150
			D347	200	부산	300
1007	마우스	9000	A210	300	광주	600
			A345	100	서울	400
			B230	200	부산	700
1201	키보드	2100	D347	200	부산	300



<제품>

제품번호	제품명	재고수량
1001	모니터	2000
1007	마우스	9000
1201	키보드	2100

<제품주문>

주문번호	제품번호	고객번호	주소	주문수량
A345	1001	100	서울	150
D347	1001	200	부산	300
A210	1007	300	광주	600
A345	1007	100	서울	400
B230	1007	200	부산	700
D347	1201	200	부산	300

해설 <주문목록> 테이블에서 반복되는 주문 관련 정보인 주문번호, 고객번호, 주소, 주문수량을 분리하면 위와 같이 제 1정규형인 <제품> 테이블과 <제품주문> 테이블이 만들어진다.

- 1차 정규화 과정으로 생성된 <제품주문> 테이블의 기본키는 (주문번호, 제품번호)이고, 다음과 같은 함수적 종속이 존재한다.

주문번호, 제품번호 → 고객번호, 주소, 주문수량
 주문번호 → 고객번호, 주소
 고객번호 → 주소

• 제 2정규형(2NF; Second Normal Form)

- 제 2정규형은 테이블 R이 제 1정규형이고, 기본키가 아닌 모든 속성이 기본키에 대하여 완전 함수적 종속을 만족하는 정규형이다.
- <주문목록> 테이블이 <제품> 테이블과 <제품주문> 테이블로 무손실 분해되면서 모두 제 1정규형이 되었지만 그 중 <제품주문> 테이블에는 기본키인 (주문번호, 제품번호)에 완전 함수적 종속이 되지 않는 속성이 존재한다. 즉 주문수량은 기본키에 대해 완전 함수적 종속이지만 고객번호와 주소는 주문번호에 의해서도 결정될 수 있으므로, 기본키에 대해 완전 함수적 종속이 아니다. 따라서 <제품주문> 테이블은 제 2정규형이 아니다.

예제 2 <제품주문> 테이블에서 주문번호에 함수적 종속이 되는 속성들을 분리하여 제 2정규형을 만드시오.

<제품주문>

주문번호	제품번호	고객번호	주소	주문수량
A345	1001	100	서울	150
D347	1001	200	부산	300
A210	1007	300	광주	600
A345	1007	100	서울	400
B230	1007	200	부산	700
D347	1201	200	부산	300

<주문목록>

주문번호	제품번호	주문수량
A345	1001	150
D347	1001	300
A210	1007	600
A345	1007	400
B230	1007	700
D347	1201	300

<주문>

주문번호	고객번호	주소
A345	100	서울
D347	200	부산
A210	300	광주
B230	200	부산

해설 <제품주문> 테이블에서 주문번호에 함수적 종속이 되는 속성인 고객번호와 주소를 분리(즉 부분 함수적 종속을 제거)해 내면 위와 같이 제 2정규형인 <주문목록> 테이블과 <주문> 테이블로 무손실 분해된다.

- 제 2정규화 과정을 거쳐 생성된 <주문> 테이블의 기본키는 주문번호이다. 그리고 <주문> 테이블에는 아직도 다음과 같은 함수적 종속들이 존재한다.

주문번호 → 고객번호, 주소
 고객번호 → 주소

• 제 3정규형(3NF; Third Normal Form)

- 제 3정규형은 테이블 R이 제 2정규형이고 기본키가 아닌 모든 속성이 기본키에 대해 이행적 함수적 종속(Transitive Functional Dependency)을 만족하지 않는 정규형이다.
- <제품주문> 테이블이 <주문목록> 테이블과 <주문> 테이블로 무손실 분해되면서 모두 제 2정규형이 되었다. 그러나 <주문> 테이블에서 고객번호가 주문번호에 함수적 종속이고, 주소가 고객번호에 함수적 종속이므로 주소는 기본키인 주문번호에 대해 이행적 함수적 종속을 만족한다. 즉 주문번호 → 고객번호이고, 고객번호 → 주소이므로 주문번호 → 주소는 이행적 함수적 종속이 된다. 따라서 <주문> 테이블은 제 3정규형이 아니다.

예제 3 〈주문〉 테이블에서 이행적 함수적 종속을 제거하여 제 3정규형을 만드시오.

〈주문〉

주문번호	고객번호	주소
A345	100	서울
D347	200	부산
A210	300	광주
B230	200	부산



〈주문〉

주문번호	고객번호
A345	100
D347	200
A210	300
B230	200

〈고객〉

고객번호	주소
100	서울
200	부산
300	광주

해설 〈주문〉 테이블에서 이행적 함수적 종속(즉 주문번호 → 주소)을 제거하여 무손실 분해함으로써 위와 같이 제 3정규형인 〈주문〉 테이블과 〈고객〉 테이블이 생성된다.

• BCNF(Boyce–Codd Normal Form)

- BCNF는 테이블 R에서 모든 결정자가 후보키(Candidate Key)인 정규형이다.
- 일반적으로 제 3정규형에 후보키가 여러 개 존재하고, 이러한 후보키들이 서로 중첩되어 나타나는 경우에 적용 가능하다.
- 아래의 〈수강_교수〉 테이블(제 3정규형)은 함수적 종속{(학번, 과목명) → 담당교수, (학번, 담당교수) → 과목명, 담당교수 → 과목명}을 만족하고 있다. 〈수강_교수〉 테이블의 후보키는 (학번, 과목명)과 (학번, 담당교수)이다.

〈수강_교수〉

학번	과목명	담당교수
211746	데이터베이스	홍길동
211747	네트워크	유관순
211748	인공지능	윤봉길
211749	데이터베이스	홍길동
211747	데이터베이스	이순신
211749	네트워크	유관순

- 〈수강_교수〉 테이블에서 결정자 중 후보키가 아닌 속성이 존재한다. 즉 함수적 종속 담당교수 → 과목명이 존재하는데, 담당교수가 〈수강_교수〉 테이블에서 후보키가 아니기 때문에 〈수강_교수〉 테이블은 BCNF가 아니다.

예제 4 〈수강_교수〉 테이블에서 결정자가 후보키가 아닌 속성을 분리하여 BCNF를 만드시오.

〈수강〉

학번	담당교수
211746	홍길동
211747	유관순
211748	윤봉길
211749	홍길동
211747	이순신
211749	유관순

〈교수〉

담당교수	과목명
홍길동	데이터베이스
이순신	데이터베이스
윤봉길	인공지능
유관순	네트워크

해설 〈수강_교수〉 테이블에서 BCNF를 만족하지 못하게 하는 속성(즉 담당교수 → 과목명)을 분리해내면 위와 같이 BCNF인 〈수강〉 테이블과 〈교수〉 테이블로 무손실 분해된다.

• 제 4정규형(4NF; Fourth Normal Form)

제 4정규형은 테이블 R에 다중 값 종속(MVD; Multi Valued Dependency) $A \twoheadrightarrow B$ 가 존재할 경우 R의 모든 속성이 A에 함수적 종속 관계를 만족하는 정규형이다.

※ 다중 값 종속(다치 종속) : A, B, C 3개의 속성을 가진 테이블 R에서 어떤 복합 속성(A, C)에 대응하는 B 값의 집합이 A 값에만 종속되고 C 값에는 무관하면, B는 A에 다중 값 종속이라 하고, $A \twoheadrightarrow B$ 로 표기함

• 제 5정규형(5NF; Fifth Normal Form)

제 5정규형은 테이블 R의 모든 조인 종속(JD; Join Dependency)이 R의 후보키를 통해서만 성립되는 정규형이다.

※ 조인 종속 : 어떤 테이블 R의 속성에 대한 부분 집합 X, Y, ..., Z가 있다고 하자. 이때 만일 테이블 R이 자신의 프로젝션(Projection) X, Y, ..., Z를 모두 조인한 결과와 동일한 경우 테이블 R은 조인 종속 JD(X, Y, ..., Z)를 만족한다고 함

340109

24.7, 21.4, 20.5, 필기 25.5, 24.2, 23.7, 20.9



059 반정규화

(A)

- 반정규화(Denormalization)는 시스템의 성능을 향상하고 개발 및 운영의 편의성 등을 높이기 위해 정규화된 데이터 모델을 의도적으로 통합, 중복, 분리하여 정규화 원칙을 위배하는 행위이다.
- 반정규화를 수행하면 시스템의 성능이 향상되고 관리 효율성은 증가하지만 데이터의 일관성 및 정합성이 저하될 수 있다.
- 과도한 반정규화는 오히려 성능을 저하시킬 수 있다.

340108

필기 23.2, 21.5, 21.3, 20.9, 20.8, 20.6



058 정규화 과정 정리

(B)

비정규 릴레이션

↓ 도메인이 원자값

1NF

↓ 부분적 함수 종속 제거

2NF

↓ 이행적 함수 종속 제거

3NF

↓ 결정자이면서 후보키가 아닌 것 제거

BCNF

↓ 다치 종속 제거

4NF

↓ 조인 종속성 이용

5NF

정규화 단계 암기 요령

두부를 좋아하는 정규화가 두부가게에 가서 가게에 있는 두부를 다 달라고 말하니 주인이 깜짝 놀라며 말했다.

두부이겠다줘? ≡ 두부이겠다조

도메인이 원자값
부분적 함수 종속 제거
이행적 함수 종속 제거
결정자이면서 후보키가 아닌 것 제거
다치 종속 제거
조인 종속성 이용

340111

필기 25.8, 25.5, 24.7, 24.5, 24.2, 23.7, 22.7, 21.5, 21.3



060 시스템 카탈로그

(B)

- 시스템 카탈로그(System Catalog)는 시스템 그 자체에 관련이 있는 다양한 객체에 관한 정보를 포함하는 시스템 데이터베이스이다.
- 시스템 카탈로그 내의 각 테이블은 사용자를 포함하여 DBMS에서 지원하는 모든 데이터 객체에 대한 정의나 명세에 관한 정보를 유지 관리하는 시스템 테이블이다.
- 카탈로그들이 생성되면 데이터 사전(Data Dictionary)에 저장되기 때문에 좁은 의미로는 카탈로그를 데이터 사전이라고도 한다.

340113

21.7, 20.5, 필기 25.8, 25.5, 25.2, 24.7, 24.5, 24.2, 23.7, 23.5, 23.2, 22.7, ...



061 트랜잭션의 특성

A

특성	의미
21.7, 20.5, 필기 25.2, 24.5, ... Atomicity(원자성)	트랜잭션의 연산은 데이터베이스에 모두 반영되도록 완료(Commit)되든지 아니면 전혀 반영되지 않도록 복구(Rollback)되어야 함
20.5, 필기 25.8, 25.5, ... Consistency (일관성)	트랜잭션이 그 실행을 성공적으로 완료하면 언제나 일관성 있는 데이터베이스 상태로 변환함
20.5, 필기 25.5, 25.2, 23.7, ... Isolation (독립성, 격리성, 순차성)	둘 이상의 트랜잭션이 동시에 병행 실행되는 경우 어느 하나의 트랜잭션 실행중에 다른 트랜잭션의 연산이 끼어들 수 없음
20.5, 필기 25.5, 25.2, 24.7, ... Durability (영속성, 지속성)	성공적으로 완료된 트랜잭션의 결과는 시스템이 고장나더라도 영구적으로 반영되어야 함

340114

필기 22.7, 20.9



062 CRUD 분석

C

- CRUD 분석은 프로세스와 테이블 간에 CRUD 매트릭스를 만들어서 트랜잭션을 분석하는 것이다.
- CRUD 분석을 통해 많은 트랜잭션이 몰리는 테이블을 파악할 수 있으므로 디스크 구성 시 유용한 자료로 활용할 수 있다.
- CRUD 매트릭스의 각 셀에는 Create(생성), Read(읽기), Update(갱신), Delete(삭제)의 앞 글자가 들어간다.

340115

필기 24.7, 21.8, 21.3



063 인덱스

C

- 인덱스(Index, 색인)는 데이터 레코드를 빠르게 접근하기 위해 <키 값, 포인터> 쌍으로 구성되는 데이터 구조이다.
- 인덱스는 레코드가 저장된 물리적 구조에 접근하는 방법을 제공한다.
- 인덱스를 통해서 파일의 레코드에 빠르게 액세스 할 수 있다.

340117

필기 25.8, 25.5, 25.2, 24.7, 24.5, 24.2, 23.7, 22.4, 20.9, 20.6



064 뷰

B

- 뷰(View)는 사용자에게 접근이 허용된 자료만을 제한적으로 보여주기 위해 하나 이상의 기본 테이블로부터 유도된, 이름을 가지는 가상 테이블이다.
- 뷰가 정의된 기본 테이블이나 뷰를 삭제하면 그 테이블이나 뷰를 기초로 정의된 다른 뷰도 자동으로 삭제된다.
- 뷰를 정의할 때는 CREATE문, 제거할 때는 DROP 문을 사용한다.

340119

필기 25.8, 25.2, 24.7, 24.5, 24.2, 23.2, 22.7, 21.5, 20.8



065 파티션의 종류

B

필기 25.8, 24.7, 24.5, ... 범위 분할 (Range Partitioning)	지정한 열의 값을 기준으로 분할함 예 일별, 월별, 분기별 등
필기 25.8, 25.2, 24.7, ... 해시 분할 (Hash Partitioning)	- 해시 함수를 적용한 결과 값에 따라 데이터를 분할함 - 특정 파티션에 데이터가 집중되는 범위 분할의 단점을 보완한 것으로, 데이터를 고르게 분산할 때 유용함 - 특정 데이터가 어디에 있는지 판단할 수 없음 - 고객번호, 주민번호 등과 같이 데이터가 고르게 걸림에 효과적임
필기 25.8, 24.7, 20.8 조합 분할 (Composite Partitioning)	- 범위 분할로 분할한 다음 해시 함수를 적용하여 다시 분할하는 방식 - 범위 분할한 파티션이 너무 커서 관리가 어려울 때 유용함

**066** 분산 데이터베이스의 목표

(B)

- 위치 투명성(Location Transparency) : 액세스하려는 데이터베이스의 실제 위치를 알 필요 없이 단지 데이터베이스의 논리적인 명칭만으로 액세스할 수 있음
- 중복 투명성(Replication Transparency) : 동일 데이터가 여러 곳에 중복되어 있더라도 사용자는 마치 하나의 데이터만 존재하는 것처럼 사용하고, 시스템은 자동으로 여러 자료에 대한 작업을 수행함
- 병행 투명성(Concurrency Transparency) : 분산 데이터베이스와 관련된 다수의 트랜잭션들이 동시에 실행되더라도 그 트랜잭션의 결과는 영향을 받지 않음
- 장애 투명성(Failure Transparency) : 트랜잭션, DBMS, 네트워크, 컴퓨터 장애에도 불구하고 트랜잭션을 정확하게 처리함

**067** RTO/RPO

(A)

20.7 RTO(Recovery Time Objective, 목표 복구 시간)	비상사태 또는 업무 중단 시점으로부터 복구되어 가동될 때까지의 소요 시간을 의미함 예) 장애 발생 후 6시간 내 복구 가능
RPO(Recovery Point Objective, 목표 복구 시점)	비상사태 또는 업무 중단 시점으로부터 데이터를 복구할 수 있는 기준점을 의미함 예) 장애 발생 전인 지난 주 금요일에 백업시켜 둔 복원 시점으로 복구 가능

**068** 임의 접근통제

(A)

- 임의 접근통제(DAC; Discretionary Access Control)는 데이터에 접근하는 사용자의 신원에 따라 접근 권한을 부여하는 방식이다.
- 데이터 소유자가 접근통제 권한을 지정하고 제어한다.
- 객체를 생성한 사용자가 생성된 객체에 대한 모든 권한을 부여받고, 부여된 권한을 다른 사용자에게 허가할 수도 있다.

**069** 강제 접근통제

(A)

- 강제 접근통제(MAC; Mandatory Access Control)는 주체와 객체의 등급을 비교하여 접근 권한을 부여하는 방식이다.
- 시스템이 접근통제 권한을 지정한다.
- 데이터베이스 객체별로 보안 등급을 부여할 수 있다.
- 사용자별로 인가 등급을 부여할 수 있다.

**070** 역할기반 접근통제

(A)

- 역할기반 접근통제(RBAC; Role Based Access Control)는 사용자의 역할에 따라 접근 권한을 부여하는 방식이다.
- 중앙관리자가 접근통제 권한을 지정한다.
- 임의 접근통제와 강제 접근통제의 단점을 보완하였다.
- 다중 프로그래밍 환경에 최적화된 방식이다.

**071** DAS

(B)

- DAS(Direct Attached Storage)는 서버와 저장장치를 전용 케이블로 직접 연결하는 방식이다.
- 일반 가정에서 컴퓨터에 외장하드를 연결하는 것이 여기에 해당된다.
- 직접 연결 방식이므로 다른 서버에서 접근할 수 없고 파일을 공유할 수 없다.



072 SAN

C

- SAN(Storage Area Network)은 DAS의 빠른 처리와 NAS의 파일 공유 장점을 혼합한 방식으로, 서버와 저장장치를 연결하는 전용 네트워크를 별도로 구성하는 방식이다.
- 파이버 채널(Fibre Channel, 광 채널) 스위치를 이용하여 네트워크를 구성한다.
- 파이버 채널 스위치는 서버와 저장장치를 광케이블로 연결하므로 처리 속도가 빠르다.
- 서버들이 저장장치 및 파일을 공유할 수 있다.



073 자료 구조의 분류

B

- 선형 구조 : 배열, 선형 리스트(연속 리스트, 연결 리스트), 스택, 큐, 덱
- 비선형 구조 : 트리, 그래프



074 스택

B

- 스택(Stack)은 리스트의 한쪽 끝으로만 자료의 삽입, 삭제 작업이 이루어지는 자료구조이다.
- 후입선출(LIFO; Last In First Out) 방식으로 자료를 처리한다.
- 저장할 기억 공간이 없는 상태에서 데이터가 삽입되면 오버플로(Overflow)가 발생한다.
- 삭제할 데이터가 없는 상태에서 데이터를 삭제하면 언더플로(Underflow)가 발생한다.

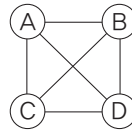


075 방향/무방향 그래프의 최대 간선 수

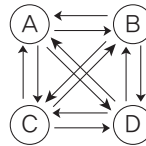
C

- 방향 그래프의 최대 간선 수 : $n(n-1)$
- 무방향 그래프에서 최대 간선 수 : $n(n-1)/2$
※ n 은 정점의 개수이다.

예제 정점이 4개인 경우 무방향 그래프와 방향 그래프의 최대 간선 수를 구하시오.



- 무방향 그래프의 최대 간선 수 : $4(4-1)/2 = 6$

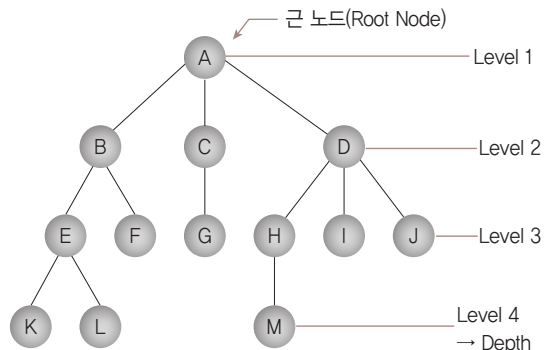


- 방향 그래프의 최대 간선 수 : $4(4-1) = 12$



076 트리 관련 용어

B



- 노드(Node) : 트리의 기본 요소로서 자료 항목과 다른 항목에 대한 가지(Branch)를 합친 것
예 A, B, C, D, E, F, G, H, I, J, K, L, M
- 근 노드(Root Node) : 트리의 맨 위에 있는 노드 예 A
- 디그리(Degree, 차수) : 각 노드에서 뻗어나온 가지의 수
예 A=3, B=2, C=1, D=3

- 단말 노드(Terminal Node) = 잎 노드(Leaf Node) : 자식이 하나도 없는 노드, 즉 Degree가 0인 노드

예 K, L, F, G, M, I, J

- Level : 근 노드의 Level을 1로 가정한 후 어떤 Level이 L이면 자식 노드는 L+1

예 H의 레벨은 3

- 깊이(Depth, Height) : Tree에서 노드가 가질 수 있는 최대의 레벨

예 위 트리의 깊이는 4

- 숲(Forest) : 여러 개의 트리가 모여 있는 것

예 위 트리에서 근 노드 A를 제거하면 B, C, D를 근 노드로 하는 세 개의 트리가 있는 숲이 생긴다.

- 트리의 디그리 : 노드들의 디그리 중에서 가장 많은 수

예 노드 A나 D가 세 개의 디그리를 가지므로 위 트리의 디그리는 3이다.

340138

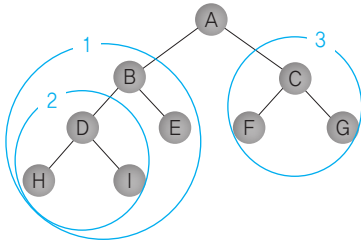


077 Preorder 운행법

(B)

Preorder 운행법은 이진 트리를 Root → Left → Right 순으로 운행하며 노드들을 찾아가는 방법이다.

Preorder 운행법의 방문 순서



※ 서브트리를 하나의 노드로 생각할 수 있도록 그림과 같이 서브트리 단위로 묶는다. 다른 운행법 모두 공통으로 사용한다.

- 1 Preorder는 Root → Left → Right이므로 A13이 된다.
- 2 1은 B2E이므로 AB2E3이 된다.
- 3 2는 DHI이므로 ABDHIE3이 된다.
- 4 3은 CFG이므로 ABDHIECFG가 된다.

• 방문 순서 : ABDHIECFG

340139

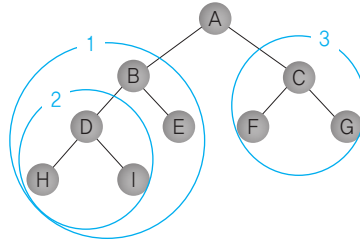


078 Inorder 운행법

(B)

Inorder 운행법은 이진 트리를 Left → Root → Right 순으로 운행하며 노드들을 찾아가는 방법이다.

Inorder 운행법의 방문 순서



- 1 Inorder는 Left → Root → Right이므로 1A3이 된다.
- 2 1은 2BE이므로 2BEA3이 된다.
- 3 2는 HDI이므로 HDIBE3이 된다.
- 4 3은 FCG이므로 HDIBEAFCG가 된다.

• 방문 순서 : HDIBEAFCG

440115

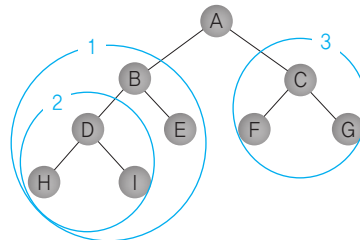


079 Postorder 운행법

(C)

Postorder 운행법은 이진 트리를 Left → Right → Root 순으로 운행하며 노드들을 찾아가는 방법이다.

Postorder 운행법의 방문 순서



- 1 Postorder는 Left → Right → Root이므로 13A가 된다.
- 2 1은 2EB이므로 2EB3A가 된다.
- 3 2는 HID이므로 HIDE3A가 된다.
- 4 3은 FGC이므로 HIDE3A가 된다.

• 방문 순서 : HIDE3A



080

Postfix로 표기된 수식을 Infix로 바꾸기

B

Postfix는 Infix 표기법에서 연산자를 해당 피연산자 두 개의 뒤로 이동한 것이므로 연산자를 다시 해당 피연산자 두 개의 가운데로 옮기면 된다.

$$ABC- / DEF+ * + \rightarrow A / (B-C) + D * (E+F)$$

- ❶ 먼저 인접한 피연산자 두 개와 오른쪽의 연산자를 괄호로 묶는다.

$$((A(BC-)) / (D(EF+)*)) +$$

- ❷ 연산자를 해당 피연산자의 가운데로 이동시킨다.

$$((A(B C -) /) (D (E F +) *) +)$$


$$((A / (B-C)) + (D * (E+F)))$$

- ❸ 필요 없는 괄호를 제거한다.

$$A / (B-C) + D * (E+F)$$


081

삽입 정렬

C

- 삽입 정렬(Insertion Sort)은 가장 간단한 정렬 방식으로, 이미 순서화된 파일에 새로운 하나의 레코드를 순서에 맞게 삽입시켜 정렬하는 방식이다.
- 평균과 최악 모두 수행 시간 복잡도는 $O(n^2)$ 이다.

예제 8, 5, 6, 2, 4를 삽입 정렬로 정렬하시오.

- 초기상태:

8	5	6	2	4
---	---	---	---	---

- 1회전:

8	5	6	2	4
---	---	---	---	---

 →

5	8	6	2	4
---	---	---	---	---

두 번째 값을 첫 번째 값과 비교하여 5를 첫 번째 자리에 삽입하고 8을 한 칸 뒤로 이동시킨다.

- 2회전:

5	8	6	2	4
---	---	---	---	---

 →

5	6	8	2	4
---	---	---	---	---

세 번째 값을 첫 번째, 두 번째 값과 비교하여 6을 8자리에 삽입하고 8은 한 칸 뒤로 이동시킨다.

- 3회전:

5	6	8	2	4
---	---	---	---	---

 →

2	5	6	8	4
---	---	---	---	---

네 번째 값 2를 처음부터 비교하여 맨 처음에 삽입하고 나머지를 한 칸씩 뒤로 이동시킨다.

- 4회전:

2	5	6	8	4
---	---	---	---	---

 →

2	4	5	6	8
---	---	---	---	---

다섯 번째 값 4를 처음부터 비교하여 5자리에 삽입하고 나머지를 한 칸씩 뒤로 이동시킨다.



082 선택 정렬

B

- 선택 정렬(Selection Sort)은 n 개의 레코드 중에서 최소값을 찾아 첫 번째 레코드 위치에 놓고, 나머지 $(n-1)$ 개 중에서 다시 최소값을 찾아 두 번째 레코드 위치에 놓는 방식을 반복하여 정렬하는 방식이다.
- 평균과 최악 모두 수행 시간 복잡도는 $O(n^2)$ 이다.

예제 8, 5, 6, 2, 4를 선택 정렬로 정렬하시오.

• 초기 상태: [8 5 6 2 4]

• 1회전: [8 5 6 2 4] → [8 5 6 2 4] → [2 5 6 8 4]
첫 번째부터 마지막 값 중 최소값 2를 찾아 첫 번째 값 8과 위치를 교환한다.

• 2회전: [2 5 6 8 4] → [2 5 6 8 4] → [2 4 6 8 5]
두 번째부터 마지막 값 중 최소값 4를 찾아 두 번째 값 5와 위치를 교환한다.

• 3회전: [2 4 6 8 5] → [2 4 6 8 5] → [2 4 5 8 6]
세 번째부터 마지막 값 중 최소값 5를 찾아 세 번째 값 6과 위치를 교환한다.

• 4회전: [2 4 5 8 6] → [2 4 5 8 6] → [2 4 5 6 8]
네 번째부터 마지막 값 중 최소값 6을 찾아 네 번째 값 8과 위치를 교환한다.



083 버블 정렬

B

- 버블 정렬(Bubble Sort)은 주어진 파일에서 인접한 두 개의 레코드 키 값을 비교하여 그 크기에 따라 레코드 위치를 서로 교환하는 정렬 방식이다.
- 평균과 최악 모두 수행 시간 복잡도는 $O(n^2)$ 이다.

예제 8, 5, 6, 2, 4를 버블 정렬로 정렬하시오.

• 초기 상태: [8 5 6 2 4]

• 1회전: [5 8 6 2 4] → [5 6 8 2 4] → [5 6 2 8 4] → [5 6 2 4 8]
첫 번째부터 마지막 값 중 최대값 8을 찾아 첫 번째 값 5와 위치를 교환한다.

• 2회전: [5 6 2 4 8] → [5 2 6 4 8] → [5 2 4 6 8]
두 번째부터 마지막 값 중 최대값 6을 찾아 두 번째 값 5와 위치를 교환한다.

• 3회전: [2 5 4 6 8] → [2 4 5 6 8]
세 번째부터 마지막 값 중 최대값 6을 찾아 세 번째 값 5와 위치를 교환한다.

• 4회전: [2 4 5 6 8]
네 번째부터 마지막 값 중 최대값 8을 찾아 네 번째 값 6과 위치를 교환한다.



084 퀵 정렬

B

- 퀵 정렬(Quick Sort)은 키를 기준으로 작은 값은 왼쪽, 큰 값은 오른쪽 서브 파일에 분해시키는 과정을 반복하는 정렬 방식이다.
- 레코드의 많은 자료 이동을 없애고 하나의 파일을 부분적으로 나누어 가면서 정렬한다.
- 평균 수행 시간 복잡도는 $O(n \log_2 n)$ 이고, 최악의 수행 시간 복잡도는 $O(n^2)$ 이다.